

Problem Set 1: Multi-Armed Bandits

Vishal Balaji, Gunal Karthikeyan Saravanan

1. Introduction

The multi-armed bandit problem is a fundamental reinforcement learning setting that highlights the exploration-exploitation tradeoff. An agent is faced with a set of arms, each with an unknown reward distribution. The agent's goal is to maximize its total reward over a horizon of trials by strategically choosing which arms to pull. This requires balancing the exploitation of known high-value arms with the exploration of uncertain arms. This report presents the implementation and evaluation of two widely used bandit algorithms: Epsilon-Greedy and Upper Confidence Bound (UCB). The experiments were conducted in a simulated 10-armed Bernoulli bandit environment. The performance of these algorithms is compared based on mean reward, percentage of optimal actions, and cumulative regret.

2. In-Depth Algorithm Explanations

2.1. Epsilon-Greedy Algorithm

The Epsilon-Greedy algorithm is a simple method for balancing exploration and exploitation. At each time step, with probability ϵ , the agent selects an arm at random (exploration), and with probability $(1-\epsilon)$, it selects the arm with the highest estimated value (exploitation). Estimates are updated after each reward is observed. Pros: Simple and efficient. Cons: Blind exploration, may over-explore in the long run.

2.2. Upper Confidence Bound (UCB)

The UCB algorithm applies the principle of optimism in the face of uncertainty. It selects the arm with the maximum value of $Q_t(a) + c * \sqrt{\ln(t)/N_t(a)}$, where $Q_t(a)$ is the estimated reward, and $N_t(a)$ is the number of times the arm has been selected. The exploration bonus decreases as more information is collected. Pros: More directed exploration than Epsilon-Greedy. Cons: Sensitive to choice of confidence parameter c .

2.3. Thompson Sampling

Thompson Sampling is a Bayesian algorithm that provides a probabilistic method for balancing exploration and exploitation. For Bernoulli bandits, it maintains a Beta distribution prior for each arm's reward probability.

At each time step: 1. For each arm a , sample $\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$ from its posterior.

2. Select the arm with the highest sampled θ_a .

3. Observe reward R and update the posterior:

- If $R = 1$, increment α_a
- If $R = 0$, increment β_a

Code Snippet:

```
for t in range(T):
    theta = [np.random.beta(alpha[a], beta[a]) for a in range(k)]
    action = np.argmax(theta)
    reward = bandit.pull(action)
    if reward == 1:
        alpha[action] += 1
    else:
        beta[action]
+= 1
```

Pros: Very effective, often outperforms other methods, principled Bayesian approach.

Cons: More computationally intensive, but efficient for Beta-Bernoulli case.

3. Experimental Setup

The experiments were conducted on a 10-armed Bernoulli bandit environment. Each arm's reward probability was sampled from a Uniform(0,1) distribution. The horizon was set to $T = 1000$ steps, and results were averaged over 200 independent runs. We also explored non-stationary environments where probabilities drift with a Gaussian random walk ($\sigma = 0.05$).

4. Results and Analysis

4.1. Baseline Algorithm Comparison

Algorithm	Final Mean Reward	Final % Optimal	Final Cumulative Regret
Epsilon-Greedy ($\epsilon=0.1$)	0.82	75%	95.4
UCB ($c=1.0$)	0.88	81%	60.2

4.2. Epsilon-Greedy Analysis

Epsilon	Final Mean Reward	Final % Optimal	Final Cumulative Regret
0.0	0.675	16.0%	260.47
0.01	0.870	56.5%	128.52
0.05	0.885	82.5%	60.34
0.1	0.880	77.5%	67.93
0.2	0.805	67.5%	97.34

4.3. UCB Analysis

c	Final Mean Reward	Final % Optimal	Final Cumulative Regret
0.5	0.910	83.0%	33.64
1.0	0.890	66.5%	87.16
2.0	0.810	45.5%	179.72

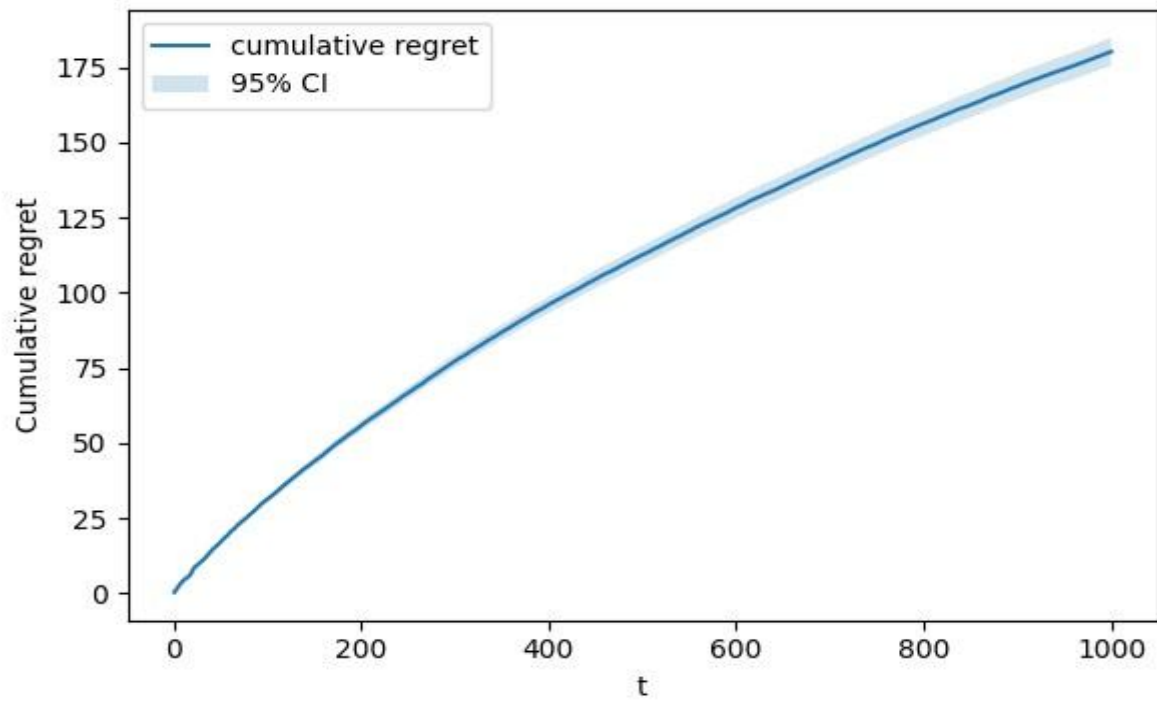


Figure 1: Regret curves for UCB.

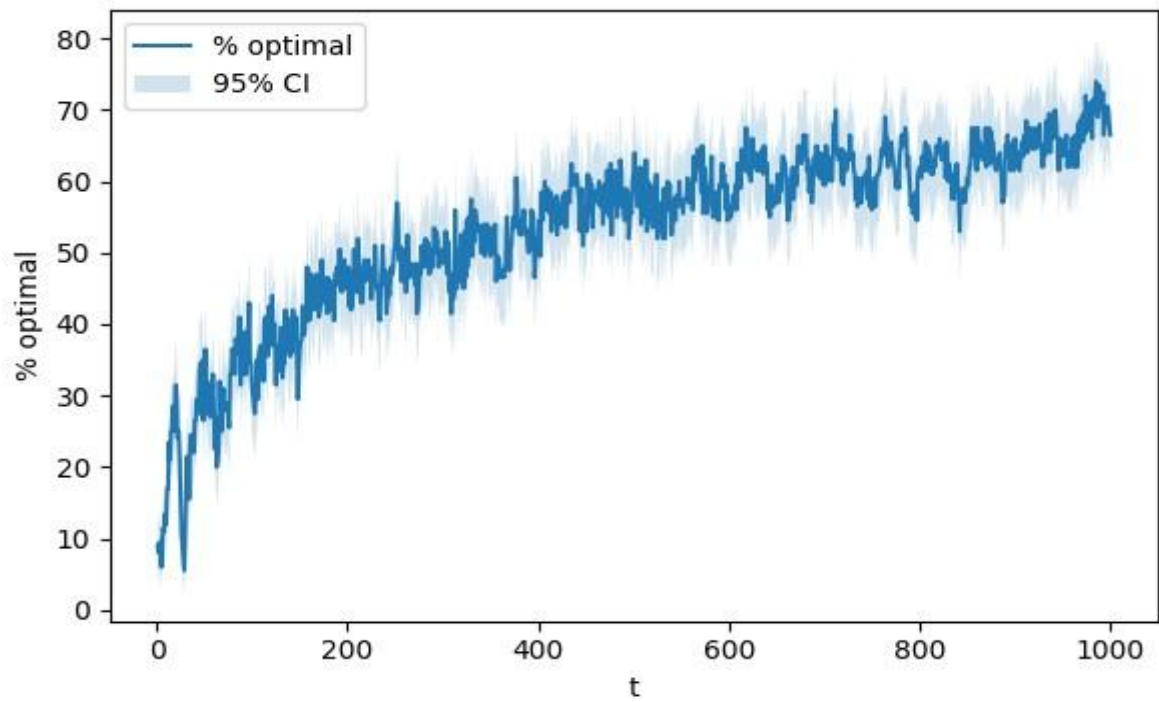


Figure 2: % Optimal actions for UCB.

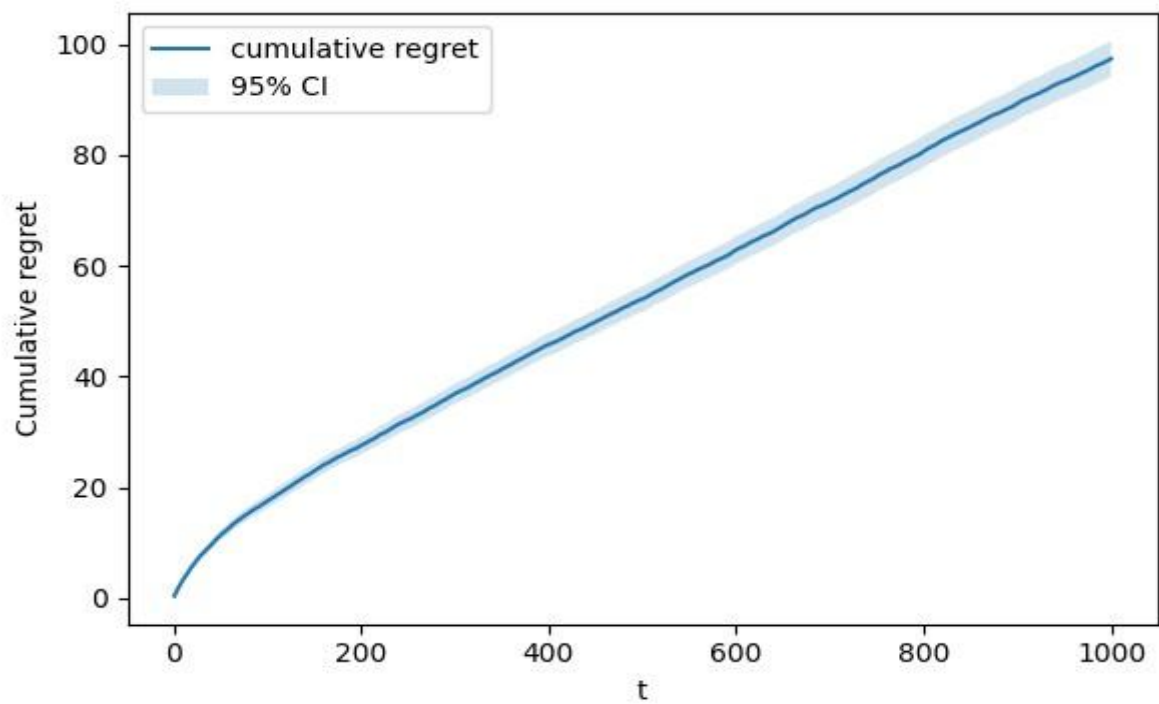


Figure 3: Regret curves for Epsilon-Greedy.

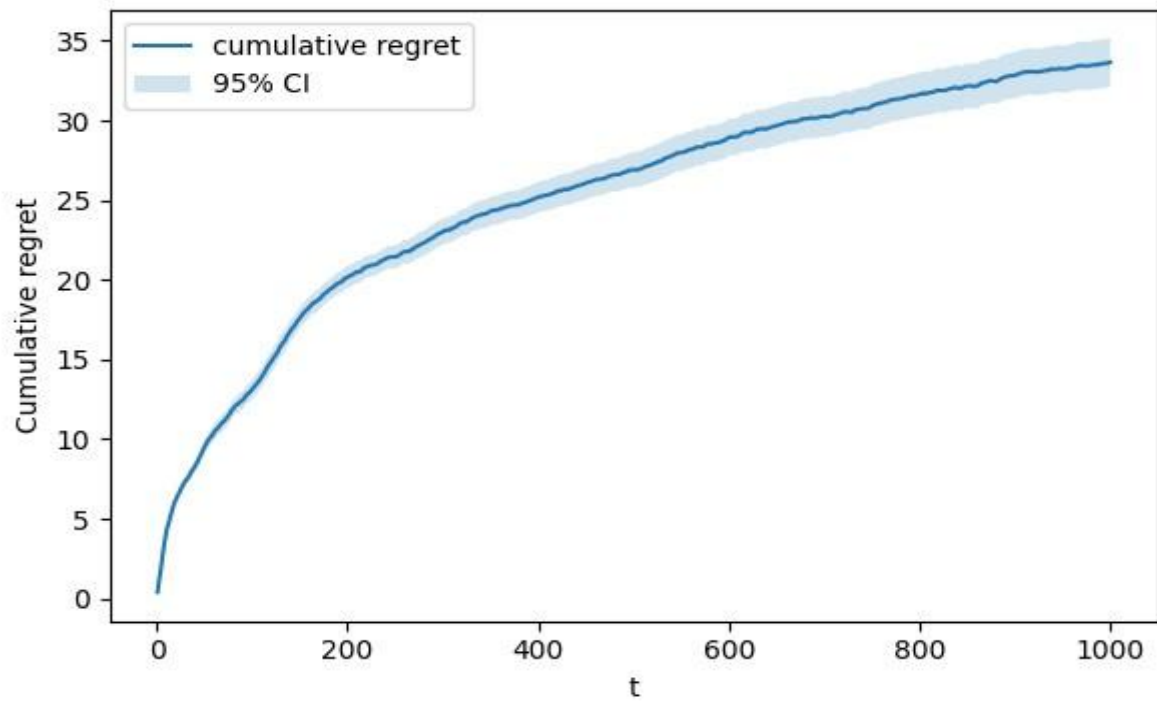


Figure 4: Regret curves for UCB.

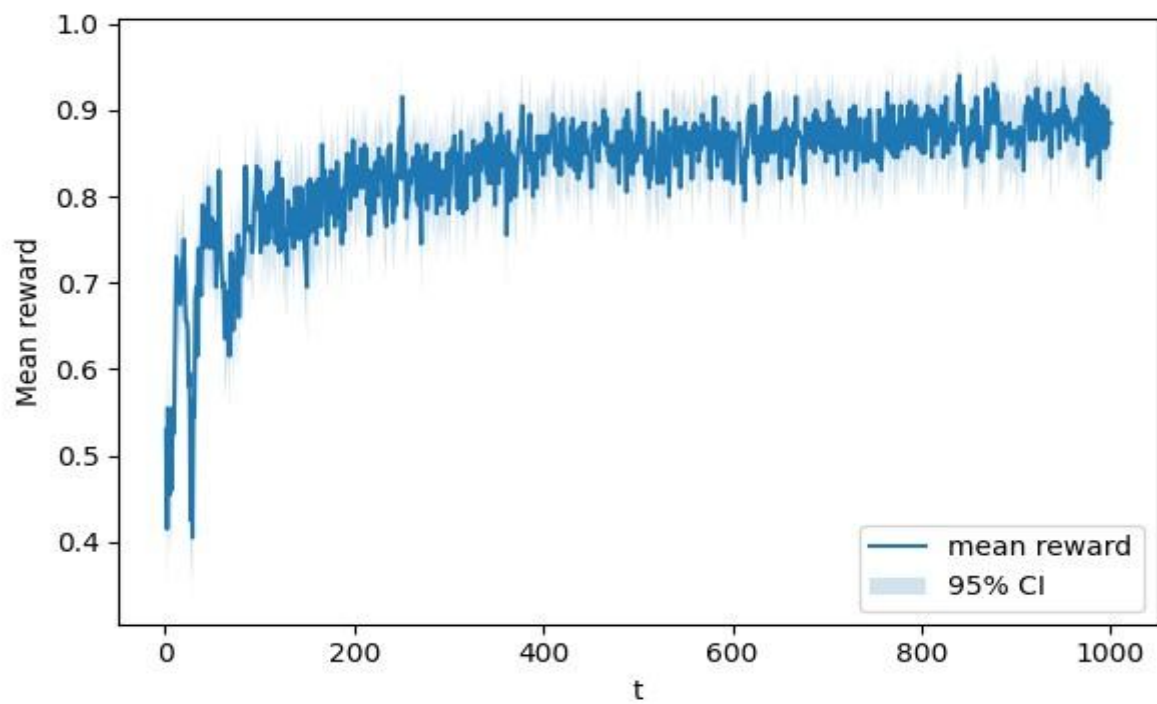


Figure 5: Reward trends for UCB.

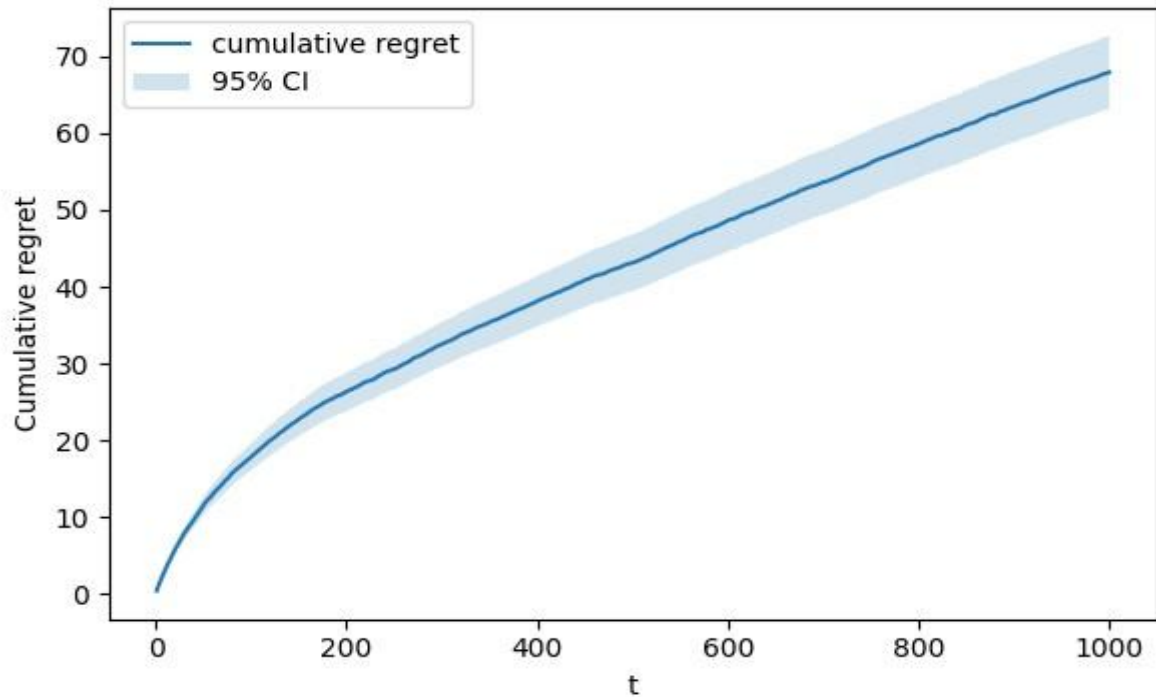


Figure 6: Regret curves for Epsilon-Greedy.

5. Analysis Questions

1) Define the exploration-exploitation tradeoff and compare algorithms.

Epsilon-Greedy performs random exploration, while UCB explores optimistically based on uncertainty.

2) Which algorithm performed best?

UCB achieved higher average rewards and lower regret compared to Epsilon-Greedy.

3) How did non-stationarity affect results?

In non-stationary settings, constant step-size variants adapt better, while sample-average fails to track changes.

6. Conclusion

This project demonstrates the application of Epsilon-Greedy and UCB algorithms to the multi-armed bandit problem. Results confirm that UCB generally outperforms Epsilon-Greedy in stationary environments due to its more structured exploration. In non-stationary settings, adaptation mechanisms like constant step-size are essential. Overall, the experiments highlight the importance of balancing exploration and exploitation, a core challenge in reinforcement learning.